

```
import cocotb
from cocotb.clock import Clock
from cocotb.triggers import RisingEdge, Timer
```

```
#
```

```
=====
```

```
=====
```

```
# RESET
```

```
#
```

```
=====
```

```
=====
```

```
async def reset_dut(dut):
```

```
    dut.rst_n.value = 0
```

```
    dut.s_axi_awaddr.value = 0
```

```
    dut.s_axi_awvalid.value = 0
```

```
    dut.s_axi_wdata.value = 0
```

```
    dut.s_axi_wstrb.value = 0xF
```

```
    dut.s_axi_wvalid.value = 0
```

```
    dut.s_axi_bready.value = 1
```

```
    dut.s_axi_araddr.value = 0
```

```
    dut.s_axi_arvalid.value = 0
```

```
dut.s_axi_rready.value = 1
```

```
if hasattr(dut, "uart_rx_pin"):
    dut.uart_rx_pin.value = 1
```

```
await Timer(100, units="ns")
```

```
dut.rst_n.value = 1
```

```
await RisingEdge(dut.clk)
```

```
#
```

```
=====
=====
```

```
# AXI WRITE
```

```
#
```

```
=====
=====
```

```
async def axi_write(dut, addr, data, strb=0xF):
```

```
    dut._log.info("")
    dut._log.info("-----")
    dut._log.info(f"WRITE ADDRESS : 0x{addr:08X}")
    dut._log.info(f"WRITE DATA   : 0x{data:08X}")
    dut._log.info("-----")
```

```
# Drive AXI write address
```

```
dut.s_axi_awaddr.value = addr
dut.s_axi_awvalid.value = 1
```

```
# Drive AXI write data
dut.s_axi_wdata.value = data
dut.s_axi_wstrb.value = strb
dut.s_axi_wvalid.value = 1
```

```
# -----
# Wait for AW/W handshake
# -----
```

```
while True:
```

```
    await RisingEdge(dut.clk)
```

```
    if dut.s_axi_awready.value and
dut.s_axi_wready.value:
        break
```

```
# Deassert VALID after handshake
dut.s_axi_awvalid.value = 0
dut.s_axi_wvalid.value = 0
```

```
# -----
# Wait for AXI write response
# -----
```

```
while True:
```

```
await RisingEdge(dut.clk)
```

```
if dut.s_axi_bvalid.value:  
    break
```

```
# -----
```

```
# IMPORTANT FIX
```

```
#
```

```
# Wait one additional clock after BVALID.
```

```
#
```

```
# This prevents the next AXI transaction from starting  
too
```

```
# close to the completion of the previous APB write.
```

```
# -----
```

```
await RisingEdge(dut.clk)
```

```
#
```

```
=====
```

```
=====
```

```
# AXI READ
```

```
#
```

```
=====
```

```
=====
```

```
async def axi_read(dut, addr):
```

```
    dut._log.info("")
    dut._log.info(f"AXI READ REQUEST  addr =
0x{addr:08X}")
```

```
# Drive AXI read address
dut.s_axi_araddr.value = addr
dut.s_axi_arvalid.value = 1
```

```
# -----
# Wait for AR handshake
# -----
```

```
while True:
```

```
    await RisingEdge(dut.clk)
```

```
    if dut.s_axi_arready.value:
        dut._log.info(
            f"AR HANDSHAKE    addr = 0x{addr:08X}"
        )
        break
```

```
# Deassert ARVALID after handshake
dut.s_axi_arvalid.value = 0
```

```
# -----
# Wait for AXI read response
# -----
```

```
while True:
```

```
    await RisingEdge(dut.clk)
```

```
    if dut.s_axi_rvalid.value:
```

```
        # Allow NBA/VPI updates to settle before  
sampling
```

```
        await Timer(1, units="ns")
```

```
        data = int(dut.s_axi_rdata.value)
```

```
        resp = int(dut.s_axi_rresp.value)
```

```
        dut._log.info(  
            f"AXI READ RESPONSE RDATA =  
0x{data:08X}"  
        )
```

```
        dut._log.info(  
            f"AXI READ RESPONSE RRESP =  
{resp:02b}"  
        )
```

```
        return data, resp
```

```
#
```

```
=====
```

```
# APB DEBUG
```

```
#
```

```
=====
```

```
def print_apb_signals(dut):
```

```
    dut._log.info("----- APB BUS -----")
```

```
    dut._log.info(
        f"PSEL    = {int(dut.m_psel.value)}"
    )
```

```
    dut._log.info(
        f"PENABLE = {int(dut.m_penable.value)}"
    )
```

```
    dut._log.info(
        f"PWRITE   = {int(dut.m_pwrite.value)}"
    )
```

```
    dut._log.info(
        f"PADDR    = 0x{int(dut.m_paddr.value):08X}"
    )
```

```
    dut._log.info(
        f"PWDATA   = 0x{int(dut.m_pwdata.value):08X}"
    )
```

```
dut._log.info("-----")
```

```
#
```

```
=====
```

```
# BASIC TEST
```

```
#
```

```
=====
```

```
@cocotb.test()
async def basic_test(dut):
```

```
    # Start clock
```

```
    cocotb.start_soon(
        Clock(dut.clk, 10, units="ns").start()
    )
```

```
    # Reset
```

```
    await reset_dut(dut)
```

```
    dut._log.info("")
```

```
dut._log.info("=====
```

```
=====")
    dut._log.info("    AXI4 -> APB -> UART
VERIFICATION")
```

```
dut._log.info("=====
=====")
    dut._log.info("")
```

#

```
=====
=====
```

ADDRESS MAP

#

```
=====
=====
```

UART_BASE = 0x40000000

TXDATA_ADDR = UART_BASE + 0x00

RXDATA_ADDR = UART_BASE + 0x04

STATUS_ADDR = UART_BASE + 0x08

BAUD_ADDR = UART_BASE + 0x0C

CTRL_ADDR = UART_BASE + 0x10

INVALID_ADDR = 0x50000000

#

```
=====
=====
```

TXDATA REGISTER TEST

#

```
=====
=====
```

```
dut._log.info("***** TXDATA TEST *****")
dut._log.info("")
```

```
tx_values = [
    0x41,
    0x55,
    0xAA,
    0x12,
    0xFF,
    0x00,
    0x7F,
    0x80,
    0x12345678,
    0xAAAAAAAA,
    0x55555555,
    0xDEADBEEF
]
```

```
for value in tx_values:
```

```
    await axi_write(
        dut,
        TXDATA_ADDR,
        value
    )
```

```
data, resp = await axi_read(  
    dut,  
    TXDATA_ADDR  
)
```

```
expected = value & 0xFF
```

```
dut._log.info(  
    f"EXPECTED DATA : 0x{expected:08X}"  
)
```

```
dut._log.info(  
    f"READ DATA    : 0x{data:08X}"  
)
```

```
if data == expected and resp == 0:
```

```
    dut._log.info(  
        "RESULT      : PASS"  
    )
```

```
else:
```

```
    dut._log.error(  
        "RESULT      : FAIL"  
    )
```

```
dut._log.info("-----")
```

#

=====

=====

BAUD REGISTER TEST

#

=====

=====

dut._log.info("")

dut._log.info("***** BAUD REGISTER TEST
*****")

dut._log.info("")

baud_values = [

0x1B,

0x10,

0x20,

0xFFFF

]

for value in baud_values:

dut._log.info(

f"BAUD WRITE DATA : 0x{value:08X}"
)

await axi_write(

dut,

```

        BAUD_ADDR,
        value
    )

    data, resp = await axi_read(
        dut,
        BAUD_ADDR
    )

    expected = value & 0xFFFF

    dut._log.info(
        f"BAUD EXPECTED : 0x{expected:08X}"
    )

    dut._log.info(
        f"BAUD READ      : 0x{data:08X}"
    )

    if data == expected and resp == 0:

        dut._log.info(
            "BAUD RESULT : PASS"
        )

    else:

        dut._log.error(
            "BAUD RESULT : FAIL"
        )

```

)

dut._log.info("")

#

=====

=====

CTRL REGISTER TEST

#

=====

=====

dut._log.info("***** CTRL REGISTER TEST
*****")

dut._log.info("")

```
ctrl_values = [  
    0x00000000,  
    0x00000001,  
    0x00000002,  
    0x0000000F,  
    0xFFFFFFFF  
]
```

for value in ctrl_values:

```
    dut._log.info(  
        f"CTRL WRITE DATA : 0x{value:08X}"
```

)

```
await axi_write(  
    dut,  
    CTRL_ADDR,  
    value  
)
```

```
data, resp = await axi_read(  
    dut,  
    CTRL_ADDR  
)
```

expected = value

```
dut._log.info(  
    f"CTRL EXPECTED : 0x{expected:08X}"  
)
```

```
dut._log.info(  
    f"CTRL READ DATA : 0x{data:08X}"  
)
```

if data == expected and resp == 0:

```
    dut._log.info(  
        "CTRL RESULT : PASS"  
    )
```

else:

```
    dut._log.error(  
        "CTRL RESULT    : FAIL"  
    )
```

```
dut._log.info("")
```

```
#
```

```
=====
```

```
=====
```

```
# STATUS REGISTER TEST
```

```
#
```

```
=====
```

```
=====
```

```
dut._log.info("***** STATUS REGISTER TEST  
*****")
```

```
dut._log.info("")
```

```
status_data, status_resp = await axi_read(  
    dut,  
    STATUS_ADDR  
)
```

```
dut._log.info(  
    f"STATUS DATA    : 0x{status_data:08X}"  
)
```

```
if status_resp == 0:
```

```
    dut._log.info(
        "STATUS READ    : PASS"
    )
```

```
else:
```

```
    dut._log.error(
        "STATUS READ    : FAIL"
    )
```

```
#
```

```
=====
```

```
=====
```

```
# INVALID ADDRESS TEST
```

```
#
```

```
=====
```

```
=====
```

```
dut._log.info("")
```

```
dut._log.info("***** INVALID ADDRESS TEST  
*****")
```

```
dut._log.info("")
```

```
dut._log.info(
```

```
    f"INVALID ADDRESS : 0x{INVALID_ADDR:08X}"
```

)

```
invalid_data, invalid_resp = await axi_read(  
    dut,  
    INVALID_ADDR  
)
```

```
dut._log.info(  
    f"INVALID RESPONSE: 0x{invalid_data:08X}"  
)
```

```
dut._log.info(  
    f"INVALID RRESP  : {invalid_resp:02b}"  
)
```

```
# APB interconnect is expected to generate:  
#  RDATA = DEAD_BEEF  
#  RRESP = SLVERR (10)
```

```
if invalid_data == 0xDEADBEEF and invalid_resp ==  
2:
```

```
    dut._log.info(  
        "INVALID ADDRESS TEST : PASS"  
    )
```

```
else:
```

```
    dut._log.error(  

```

```
    "INVALID ADDRESS TEST : FAIL"  
)
```

```
#
```

```
=====
```

```
=====
```

```
# END
```

```
#
```

```
=====
```

```
=====
```

```
    dut._log.info("")
```

```
dut._log.info("=====
```

```
=====")
```

```
    dut._log.info("        SIMULATION FINISHED")
```

```
dut._log.info("=====
```

```
=====")
```